

CI/CD Tooling & Practices – Part 3

WELCOME TO CLASS 9!



Goals of the day



Continuous Delivery



Jenkins & Ansible



Deployment Patterns



Photo by [David Becker](#) on [Unsplash](#)

©Maja Bjeletic@unsplash

Agenda

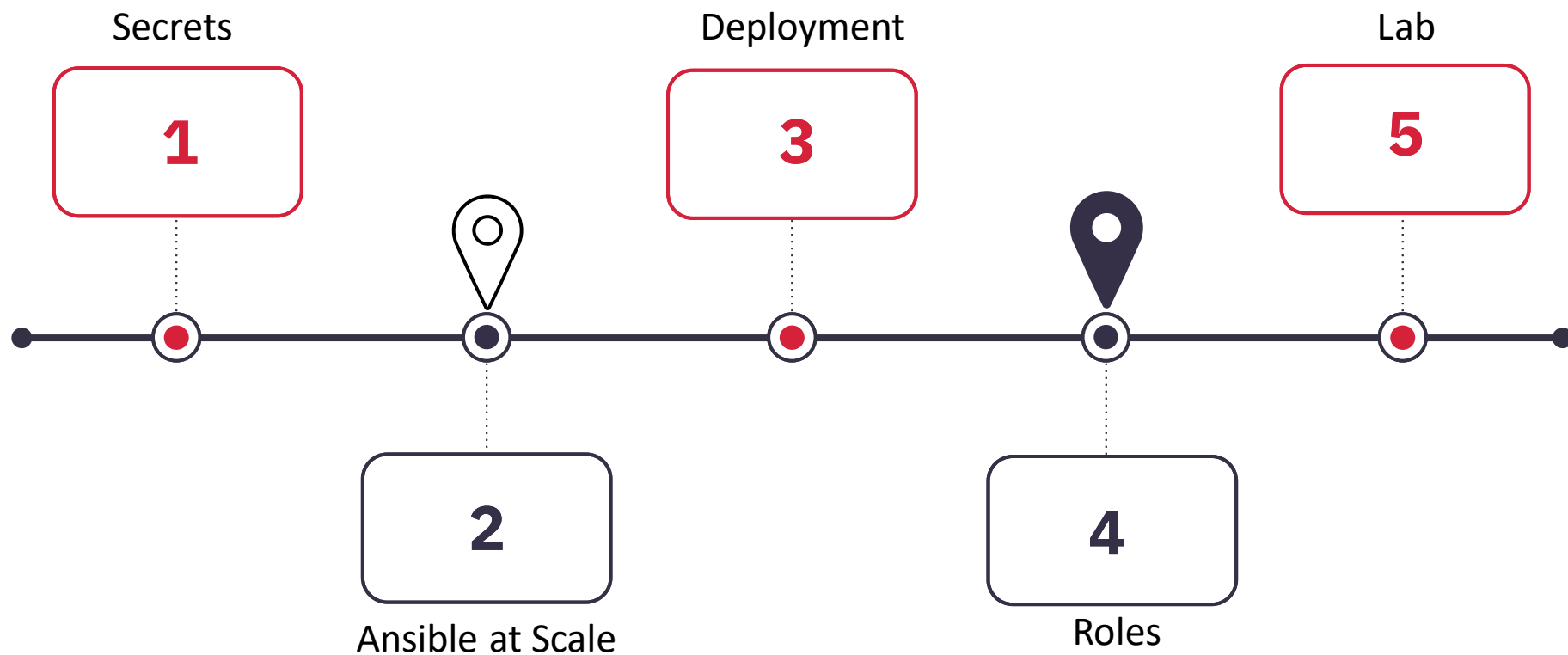
- Continuous Delivery
- **Break**
- Jenkins and Continuous Delivery
- Ansible and software deployment
- Deployment patterns
- Jenkins Groovy Libraries

Photo by David Becker on Unsplash

Review

- What did you learn about Continuous Integration in the last class?
- Which practices should you include in a Jenkins pipeline to adhere to Continuous Integration principles?
- How is Continuous Integration beneficial to development teams and their organizations?
- What do you know about Continuous Deployment?

Journey Map



Next, Jenkins Groovy Libraries

Jenkins Groovy Libraries (1 of 5)

One of the benefits of Jenkins is how customizable it is through code. With library code, frequent operations that are common across multiple pipelines can be efficiently managed.

- Consider API deployments, you may have hundreds of microservices, all slightly different.
- Do you have a pipeline for each one or a framework of libraries that are used by very light, API specific, pipelines to achieve a consistent delivery across your estate?
- This library framework becomes software written in Groovy and embedded in Jenkins.

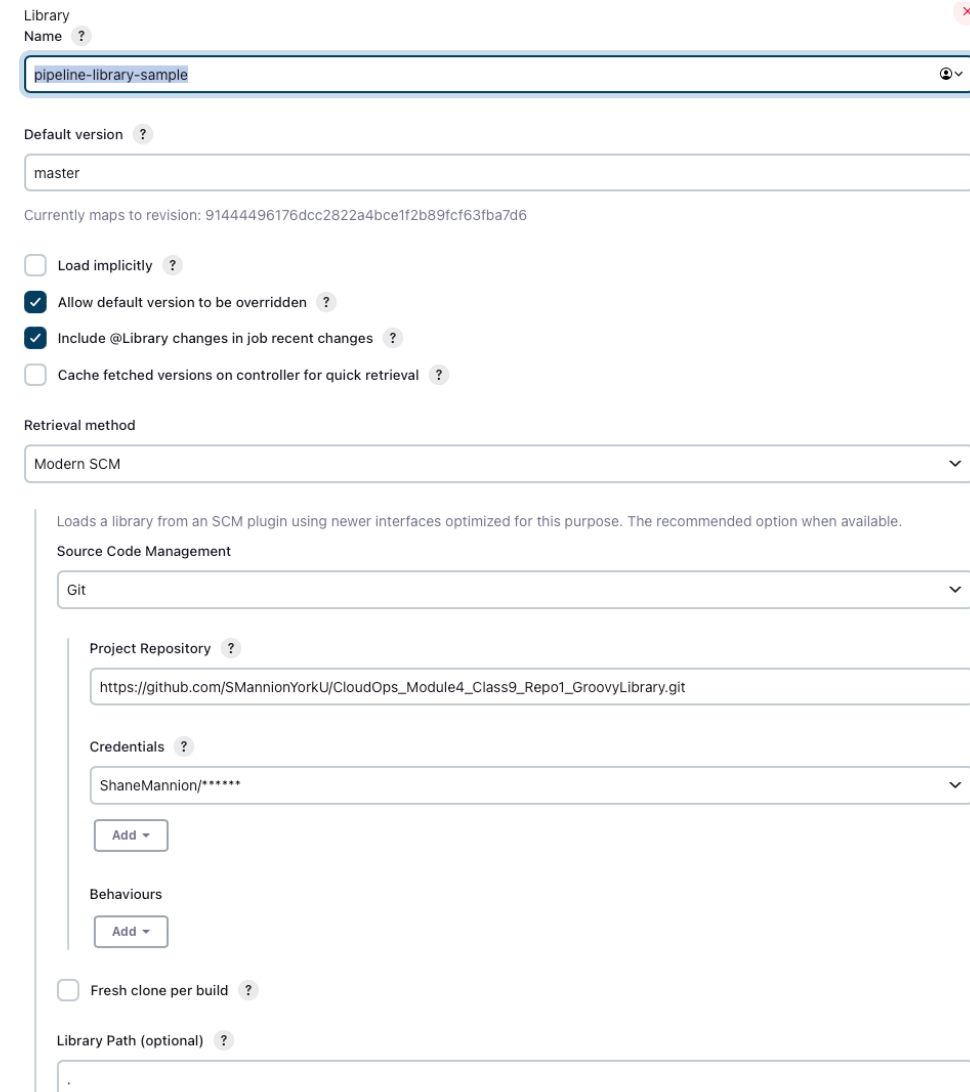


Image by Dall-E

Jenkins Groovy Libraries (2 of 5)

First step in this process is to add a library, then we need an application pipeline job to use that library.

1. We have GitHub credentials configured from the last class.
2. Navigate through Dashboard > Manage Jenkins> System and search for 'Global Pipeline Libraries'.
3. Add a new library with name 'pipeline-library-sample'. Specify master as default version. Add a link to your clone of the library code.
4. Click Save once configuration complete.



The screenshot shows the Jenkins 'Add Library' configuration page. The 'Library Name' field is set to 'pipeline-library-sample'. The 'Default version' is set to 'master'. Below this, there are four checkboxes: 'Load implicitly' (unchecked), 'Allow default version to be overridden' (checked), 'Include @Library changes in job recent changes' (checked), and 'Cache fetched versions on controller for quick retrieval' (unchecked). The 'Retrieval method' is set to 'Modern SCM'. A note states: 'Loads a library from an SCM plugin using newer interfaces optimized for this purpose. The recommended option when available.' Under 'Source Code Management', 'Git' is selected. The 'Project Repository' field contains the URL 'https://github.com/SMannionYorkU/CloudOps_Module4_Class9_Repo1_GroovyLibrary.git'. The 'Credentials' field is set to 'ShaneMannion/*****'. There are 'Add' buttons for 'Behaviours' and 'Fresh clone per build' (which is unchecked). The 'Library Path (optional)' field is empty.

Jenkins Groovy Libraries (3 of 5)

Next step is to create a build job for a modified version of the Spring Boot app from the last class, based on new repo URL above.

1. Create a pipeline job in Jenkins, with details as per the screenshot. Click Save.
2. Run the job and review the output.
3. If the job runs successfully then we can review the library code and observe its output in the job console.

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/SMannionYorkU/CloudOps_Module4_Class9_Repo2_SpringBootLibrarySample.git

Credentials ?

ShaneMannion/*****

Add

Advanced

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/master

Add Branch

Jenkins Groovy Libraries (4 of 5)

```
1  #!/usr/bin/env groovy
2  package com.yorku
3
4  def outputReport(){
5      def title = "\t" + "Jenkins build report for " + env.BUILD_TAG + "\n"
6      def buildDetails = "  Jenkins job '" + env.JOB_BASE_NAME + "' was executed by user:" + env.USER + ".\n"
7      def nodeDetails = "  The job executed on node " + env.NODE_NAME + ".\n"
8      def workspaceDetails = "  The workspace was " + env.WORKSPACE + ".\n"
9      def serverDetails = "  Jenkins login URL " + env.HUDSON_URL + ".\n"
10     def border = "*****" + "\n"
11
12     println (border + border + title + border + buildDetails + nodeDetails + workspaceDetails + serverDetails + border + border)
13 }
```

- The code formats some environment values which every Jenkins job will have and outputs them in a standard format.
- This gives consistency to output for reporting purposes, which could be processed based on this format through an observability platform or this code could be replaced with a write to a database to log important details for every job so that information can be reviewed easily.

Jenkins Groovy Libraries (5 of 5)

```
*****
*****
Jenkins build report for jenkins-SpringBootLibraryPipeline-3
*****
Jenkins job 'SpringBootLibraryPipeline' was executed by user:java-node.
The job executed on node java-node.
The workspace was /home/java-node/Jenkins/workspace/SpringBootLibraryPipeline.
Jenkins login URL http://54.208.129.16:8080/.
*****
*****
```

- The Jenkins output aligns with the library format, as expected.
- This library could be added to our Python pipeline too to generate consistency across CI pipelines for different tech stacks.
- Other options are to centralize calls to a security scanning tool for your application code, so that all calls are from a centrally maintained block of code, instead of each application establishing their own connection to the tool. In the event of a change in the interface due to an upgrade etc., having one place to update is much better than having many.

Continuous Delivery

Continuous Delivery

- Now that we understand Continuous Integration and development teams are working harmoniously to commit code regularly, automate the build, package, scan and test of their application, what are we going to do with the finished product?
- Should we let it stockpile all this great development work and then do a production release once per month or once per quarter?
- No, we should figure out how to do Continuous Delivery!

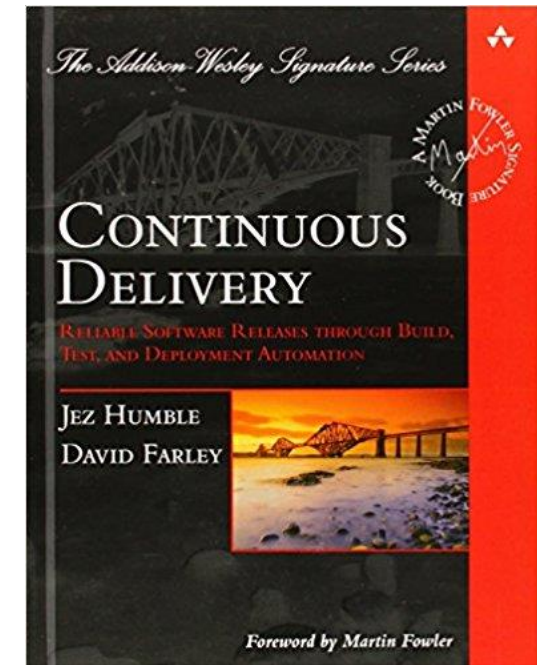


Photo by [Rowan Freeman](#) on [Unsplash](#)

Continuous Delivery [from class 7]

This is achieved through the following practices:

- Keep everything in version control
- Build once, deploy many times
- Deploying to production like environment early in the development lifecycle
- Automated environment configuration management
- Changes trigger feedback quickly, which is acted upon
- Done means released
- Approval required to promote code to Production.



This Photo by Unknown Author is licensed under
CC BY-NC-ND

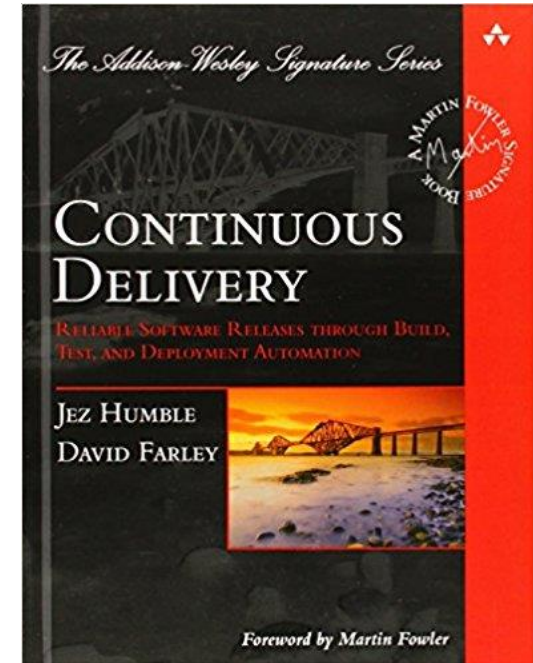
Continuous Delivery Principles (1 of 5)

- **Keep everything in version control**

we normally think in terms for application code when we think about version control. Everything means, IaC, test automation scripts, documentation, database scripts – everything you need to stand up and operate your application should be in version control

- **Build once, deploy many times**

you build or package the code for a DEV deployment, should you also build for Production? No! Build once and deploy that artifact to each successive region, all the way to Production. That creates confidence that the code deployed to Production works across many environments. The application needs to be designed to work in this way. If you need to add environment specific config into every build e.g. database url, then your application breaks this principle.



This Photo by Unknown Author is licensed under
CC BY-NC-ND

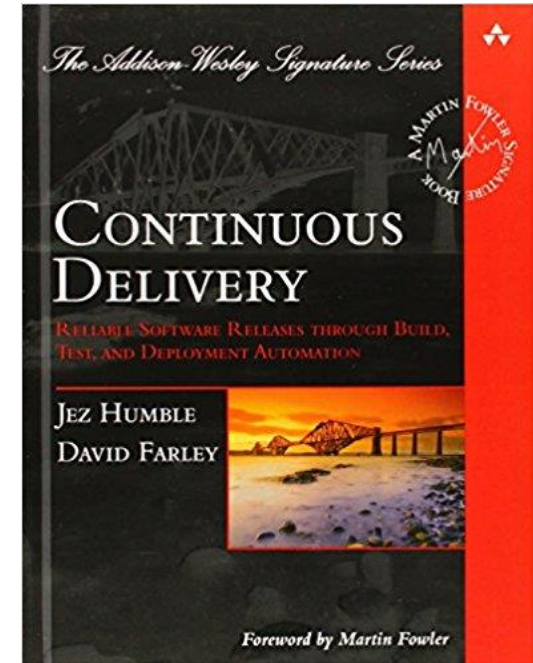
Continuous Delivery Principles (2 of 5)

- **Deploying to production like environment early in the development lifecycle**

Each successive environment should be closer to Production in terms of resources, sizing, configuration, etc so that testing is representative of how your application will run in Production. Using IaC to maintain parity between environments is an excellent way to enforce this principal

- **Automated environment configuration management**

When environments are configured manually, it introduces variances across regions. Unexpected outcomes are observed, code that was tested successfully ends up with defects in Production. Automating configuration management and putting appropriate controls in place around updates prevents these types of issues.



This Photo by Unknown Author is licensed under
CC BY-NC-ND

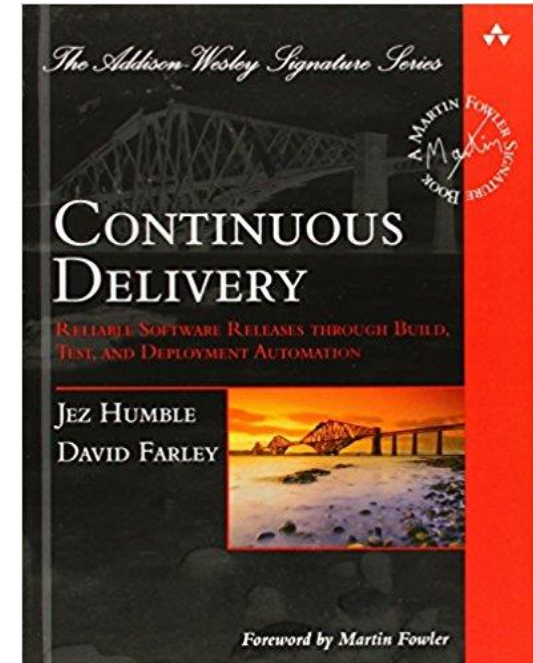
Continuous Delivery Principles (3 of 5)

- **Changes trigger feedback quickly, which is acted upon**

When the build breaks – take action! Because code is delivered continuously, if there is a problem deployment, it quickly impacts other developers. When a deployment fails, the person responsible for that release should drop everything and try to fix it. They should seek help if needed. After an hour if the issue is not resolved, the team should swarm on the issue, as it is preventing progress for everyone, until the issue is resolved. Fast feedback is one of the Three Ways of DevOps.

- **Done means released**

This means released into Production. At that time, there is no more testing, merging, regression – the change is in production and the team can move on to the next change. This also implies that getting something done is a group effort – no one person can do so alone.



This Photo by Unknown Author is licensed under
CC BY-NC-ND

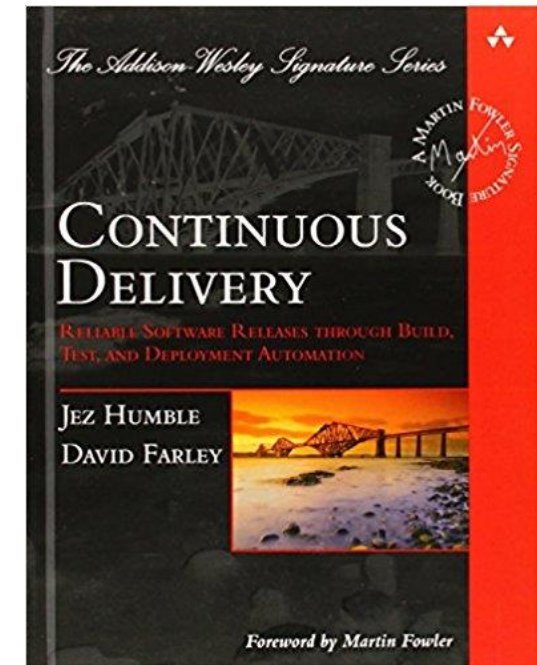
Continuous Delivery Principles (4 of 5)

- **Approval required to promote code to Production.**

Unlike Continuous Deployment, where the code is pushed straight to Production, Continuous Delivery has an approval step before that final automated deployment.

- **Continuous Improvement**

The first automated deployment of an application will generally not be fully optimized. Having a focus on Continuous Improvement means that the team is constantly looking for opportunities to make the process better. This idea is called kaizen in the Toyota Production System and is one of the principals of Lean.



This Photo by Unknown Author is licensed under
CC BY-NC-ND

Continuous Delivery Practices (5 of 5)

- Don't check in on a broken build
- Always run all commit tests locally before committing
- Wait for the commit tests to pass before moving on
- Never go home on a broken build
- Always be prepared to revert to the previous version
- Time-box fixing before reverting
- Don't comment out failing tests
- Take responsibility for all breakages that result from your changes
- Test driven development



Photo by [Jungwoo Hong](#) on [Unsplash](#)

Jenkins & Continuous Delivery

Jenkins & Continuous Delivery (1 of 5)

At this point you are very familiar with Jenkins as a Continuous Integration tool, however it is equally as adept in the Continuous Delivery space.

It's vast ecosystem of plugins, the ability to codify everything and check that code in alongside the application code in an SCM and its unparalleled open-source community make it a great choice for CD.

Although it is Open Source and competes with equivalent products like Azure DevOps (from Microsoft), it still has the largest market share for CICD tooling.



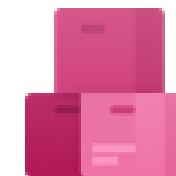
This Photo by Unknown Author is licensed under CC BY-SA

Jenkins & Continuous Delivery (2 of 5)

CI pipelines typically build/package the code, run unit tests, scan code and run some integration tests.

A common practice for CI is to store the built/packaged artifact rather than deploy it right away. This creates a separation between the CI and CD pipelines.

There are many products available to provide this versioned storage facility, including those shown on this slide.



Azure Artifacts

Jenkins & Continuous Delivery (3 of 5)

Versioning of software is key concept in storing built artifacts.

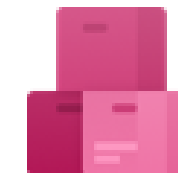
The semantic versioning convention for software is widely adopted. This is structured as follows:

$x.y.z$

where

x	major version
y	minor version
z	patch release

where



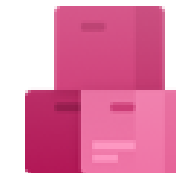
Azure Artifacts

Jenkins & Continuous Delivery (4 of 5)

Artifact storage is company specific - your organization will have its own Nexus or Artifactory instance.

The version of the application being built is stored in the pom.xml for Java applications. If your company artifact repository already has an existing version of your built software, then attempts to push a different build with the same version number will fail (unless in the case of maven projects, you use `-SNAPSHOT`).

With each change being a potential release candidate, having a clear strategy on versioning is important.



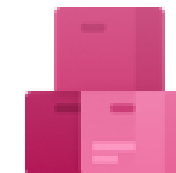
Azure Artifacts

Jenkins & Continuous Delivery (5 of 5)

Our next step is to revisit the Spring Boot App that we built in the last class.

To keep this simple, the sample code for this exercise will not use a dedicated build node, we will just build on the server.

Rather than add an Artifact repository, which will add complexity, we will store the built artifact on the Jenkins server file system in the CI job and then provide a path to that location in the CD job.



Azure Artifacts

Jenkins Deployment Pipeline using Ansible Playbook

CICD for Python Application (incl Ansible deployment)

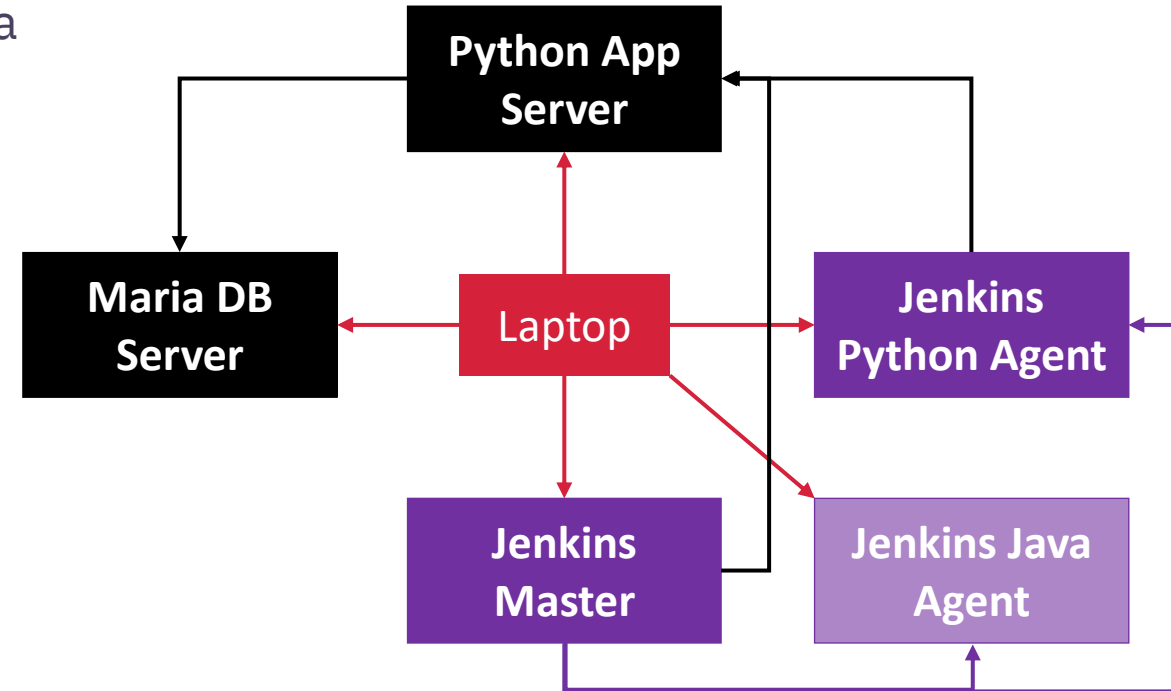
(1 of 16)

In this exercise we will build and deploy the Python API we reviewed in class8. We will also connect the app to a SQL instance and verify that it all works by calling the API using curl.

There are 5 servers running in total

- 3 pre-existing Jenkins servers: 1 master & 2 agents [Java agent not used here]
- 1 Maria DB server
- 1 Python Application server

In the middle is the laptop, which will need to connect to all these servers over SSH, while connections between these nodes are also established.



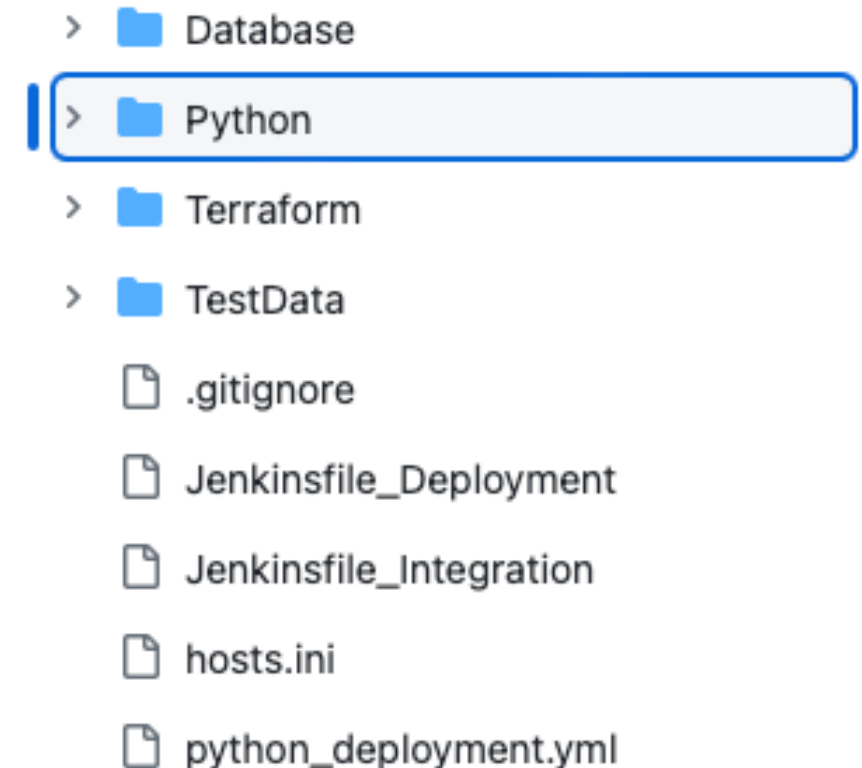
CICD for Python Application (incl Ansible deployment)

(2 of 16)

https://github.com/SMannionYorkU/CloudOps_Module4_Class9_Repo3_Python_ContinuousDelivery.git

To start, review the code in this repo.

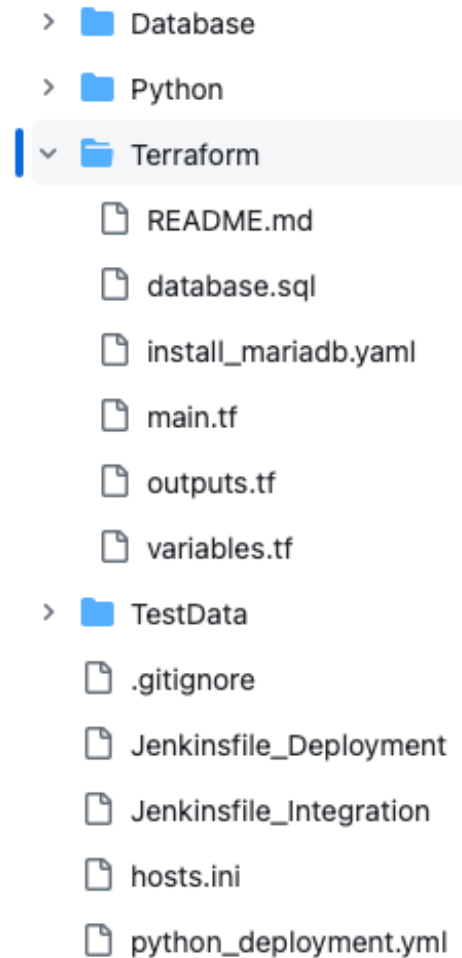
1. The codebase structure should be familiar from the last class, but there are some new files
 - JenkinsFile_Integration contains lint, test and packaging steps
 - JenkinsFile_Deployment copies the packaged code and pushes it to the Python App Server, where it is configured and started
 - python_deployment.yml is an Ansible playbook used to support the deployment process



CICD for Python Application (incl Ansible deployment)

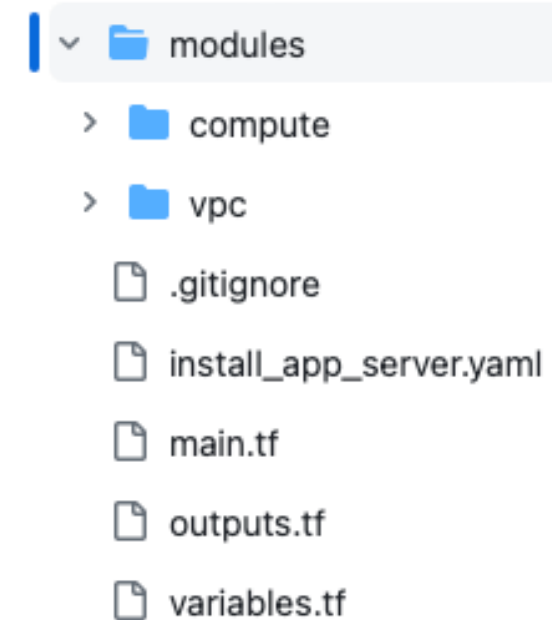
(3 of 16)

2. The Terraform folder contains the code to create, configure and load the MariaDB SQL instance.
 - This is very similar to the code we reviewed in class 5. The key difference is the database.sql file that contains statements to create and load data across multiple tables.
 - Clone this repo to your laptop and create this database server, noting the server IP.



CICD for Python Application (incl Ansible deployment) (4 of 16)

3. The final server to mention is the server where the Python API will be deployed and run. This code is in a separate repo.
- It uses the familiar module structure, including an Ansible playbook which creates the python-app-server user and installs python/pip
 - The EC2 is created using a 2023 Amazon Linux AMI, as it comes with Python 3.9 preinstalled, which the API is built on.
 - Clone this repo to your laptop and create this application server, noting the server IP.



CICD for Python Application (incl Ansible deployment) (5 of 16)

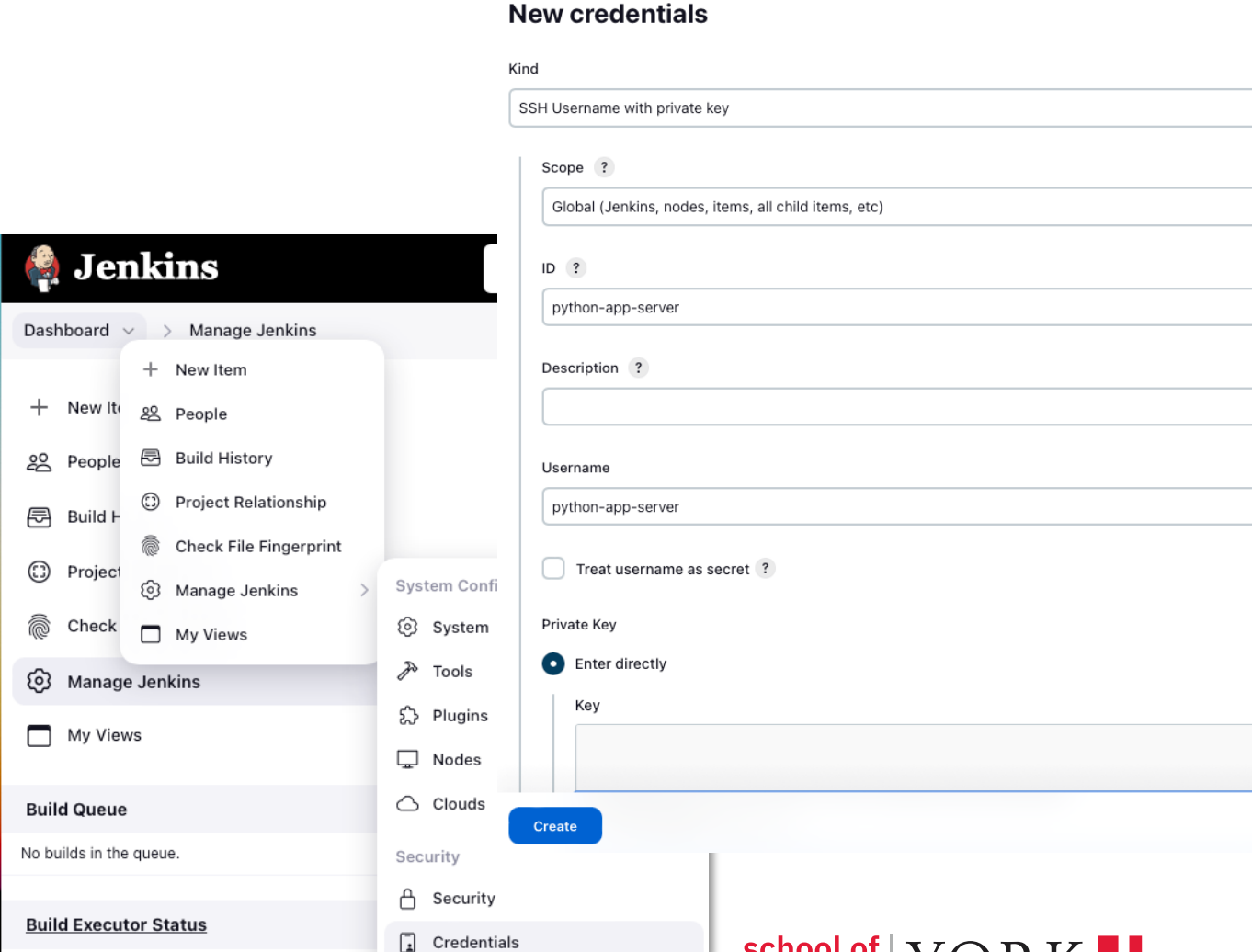
4. Connect to the Python App server and configure SSH:

- a) `ssh ec2-user@<app server IP>`
- b) `sudo su - python-app-server`
- c) `ssh-keygen -t ed25519 -N "" -f /home/python-app-server/.ssh/id_ed25519`
- d) `cd .ssh && cat id_ed25519.pub > authorized_keys`
- e) `chmod 700 authorized_keys`
- f) `cat id_ed25519 <you will use this key to add to create a credential in Jenkins, which will be used to move files and run a playbook on this app server>`

```
[ec2-user@ip-10-0-1-111 ~]$ sudo su - python-app-server
[python-app-server@ip-10-0-1-111 ~]$ ssh-keygen -t ed25519 -N "" -f /home/python-app-server/.ssh/id_ed25519
Generating public/private ed25519 key pair.
Created directory '/home/python-app-server/.ssh'.
Your identification has been saved in /home/python-app-server/.ssh/id_ed25519
Your public key has been saved in /home/python-app-server/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:QbBGP+q/9h77nCBhqNm5DLVzRM9CkIcGGpRxPHCIUg python-app-server@ip-10-0-1-111.ec2.internal
The key's randomart image is:
+--[ED25519 256]--+
|.Eo+**=0.
| .  =0=+0  .
|  o o.=.o . o
| .  o o.* + .
|   o S.B .
|  + . o.. .
| o =  .. .
|   = .o o.
|   .oo.+ =0
+-----[SHA256]-----+
[python-app-server@ip-10-0-1-111 ~]$ cd .ssh && cat id_ed25519.pub > authorized_keys
[python-app-server@ip-10-0-1-111 .ssh]$ chmod 700 authorized_keys
```

CICD for Python Application (incl Ansible deployment) (6 of 16)

5. Next create the credential to connect to the Python app server from Jenkins.
6. Go to Credentials, select 'Add New Credential', select 'SSH Username with private key'.
7. Add python-app-server as ID and username and copy the private key from the previous `cat id_e25519` command.



The screenshot displays the Jenkins web interface. On the left, the sidebar shows the 'Manage Jenkins' menu with a dropdown open, highlighting 'New Item'. The main content area is titled 'New credentials' and contains the following fields:

- Kind:** SSH Username with private key
- Scope:** Global (Jenkins, nodes, items, all child items, etc)
- ID:** python-app-server
- Description:** (empty field)
- Username:** python-app-server
- Treat username as secret:** ☐
- Private Key:** Enter directly (selected)
- Key:** (empty text area)
- Create:** (blue button)

The sidebar also shows the 'Build Queue' and 'Build Executor Status' sections.

CICD for Python Application (incl Ansible deployment) (7 of 16)

- Next go back to the command/putty window that is connected to the Python Agent (not App Server) and locate the Ansible installation.
- In Jenkins, under 'Manage Jenkins', select 'Plugins'. Click on Available Plugins, enter Ansible and install.

```
[[python-node@ip-10-0-1-99 ~]$ which ansible
/usr/bin/ansible
[[python-node@ip-10-0-1-99 ~]$
```

Dashboard > Manage Jenkins > Plugins

Updates
Available plugins
Installed plugins
Advanced settings
Download progress

Plugins

Search available plugins

Install

Install	Name ↓	Released
☒	Ansible 276.vcf26df37652f pipeline External Site/Tool Integrations DevOps Build Tools Deployment Invoke Ansible Ad-Hoc commands and playbooks.	17 days ago
☐	Ansible Tower 0.16.0 This plugin connects Jenkins with Ansible Tower	3 yr 4 mo ago

CICD for Python Application (incl Ansible deployment) (8 of 16)

10. Next under 'Manage Jenkins', select 'Tools'. Click on 'Add Ansible' at the end of the page.

11. Enter ANSIBLE as the name and for the path, use the path from the which ansible command, but omitting ansible from the end e.g. if **which ansible** returned **/usr/bin/ansible** then enter **/usr/bin/**

12. Click Save.

Ansible installations

Add Ansible

≡ Ansible

Name

ANSIBLE

Path to ansible executables directory

/usr/bin/

☐ Install automatically ?

CICD for Python Application (incl Ansible deployment) (9 of 16)

13. Now the pipeline creation can start. Create a new pipeline job, called Python_Integration.

14. Add the Repo3_Python_ContinuousDelivery URL and for the 'Script Path', change to Jenkinsfile_Integration.

15. Save the job and click on 'Build Now'.

Pipeline

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/SMannionYorkU/CloudOps_Module4_Class9_Repo3_Python_ContinuousDelivery.git

Credentials ?

ShaneMannion/*****

Add

Advanced

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/master

Add Branch

Repository browser ?

(Auto)

Additional Behaviours

Add

Script Path ?

Jenkinsfile_Integration

CICD for Python Application (incl Ansible deployment)

(10 of 16)

16. As an overview of the `JenkinFile_Integration` file:

- Runs on Python-Node
- Runs Pylint on the python code, use switch **--exit-zero** as any non-zero return code returned by Pylint will be registered as a failure by Jenkins
- Run Pytest – if a test fails, the pipeline will fail
- Package the code (removing files not required at runtime), zip the code and copy it to a folder. The zip file will be retrieved from here by the deployment job.

```
pipeline {
  agent {
    node {
      label 'Python-Node'
    }
  }
  stages {
    stage('Lint') {
      steps {
        dir('Python') {
          sh 'pip3 install -r requirements.txt'
          sh 'pylint *.py --exit-zero'
        }
      }
    }
    stage('Test') {
      steps {
        dir('Python') {
          sh 'pytest'
        }
      }
    }
    stage('copy code for deployment'){
      steps {
        dir('Python') {
          sh 'rm test*.py'
          sh 'rm packages.*'
        }
        sh 'zip -r gradesApp.zip Python'
        sh 'mkdir -p ~/python_deployment && cp ./gradesApp.zip ~/python_deployment'
      }
    }
  }
  post {
    success {
      echo 'Job completed successfully'
    }
    failure {
      echo 'Job failed'
    }
  }
}
```

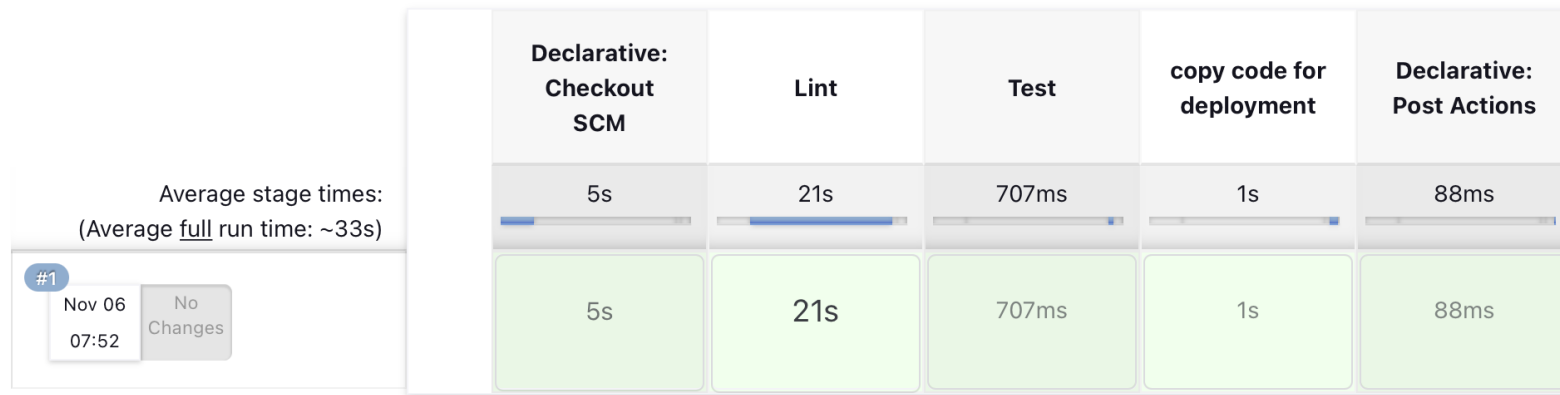
CICD for Python Application (incl Ansible deployment)

(11 of 16)

17. This pipeline is very similar to the CI Pipeline for Python API from class 8, with the addition of the archiving of the code at the end.

- In practice, the packaged code would be version and pushed to a secure artifact storage solution, such as JFrog Artifactory or Sonatype Nexus.

Stage View



CICD for Python Application (incl Ansible deployment)

(12 of 16)

18. Now the deployment pipeline can be created. Add a new pipeline job, the same as Python_Integration called Python_Deployment, but with the 'Script Path', changed to Jenkinsfile_Deployment.

19. Reviewing the Jenkinsfile_Deployment file

- Accepts parameters for app server and db server as user input. App server specifies where the code will be deployed and run. DB server is mapped into application configuration
- Copy the zip file created in the integration job to the app server and unzip
- Runs the Ansible playbook, details on next slide
- Runs the Python API

```
pipeline {
    agent {
        node {
            label 'Python-Node'
        }
    }

    parameters {
        string(name: 'APP_NODE_IP', description: 'IP address of Python App Server', defaultValue: '33.33.33.33')
        string(name: 'DB_NODE_IP', description: 'IP address of DB Server', defaultValue: '44.44.44.44')
    }

    stages {
        stage('SCP copy') {
            steps {
                withCredentials([sshUserPrivateKey(credentialsId: 'python-app-server', keyFileVariable: 'MY_SSH_KEY')]) {
                    sh "echo '${params.APP_NODE_IP}' >> hosts.ini"
                    sh "ssh -o StrictHostKeyChecking=no -i $MY_SSH_KEY python-app-server@${APP_NODE_IP} \"mkdir -p ~/deployments\""
                    sh "scp -i $MY_SSH_KEY /home/python-node/python_deployment/gradesApp.zip python-app-server@${APP_NODE_IP}:~/deployments"
                }
            }
        }

        stage('run Ansible Playbook') {
            steps {
                // running this 1st (commented out) command locally on the Python node (with the public key from that node
                // added to the app server authorized keys) is a good way to confirm that the Ansible part is OK, before layering on Jenkins,
                // sh 'ansible-playbook -i hosts.ini python_deployment.yml --extra-vars \"database_ip=${DB_NODE_IP}\"'
                // using the ansiblePlaybook is better though as it uses the Ansible instance used in the setup and Jenkins managed credentials
                ansiblePlaybook(
                    credentialsId: 'python-app-server',
                    disableHostKeyChecking: true,
                    extraVars: [database_ip: "${DB_NODE_IP}"],
                    installation: 'ANSIBLE',
                    inventory: 'hosts.ini',
                    playbook: 'python_deployment.yml',
                    vaultTmpPath: ''
                )
            }
        }

        stage('run Application') {
            steps {
                withCredentials([sshUserPrivateKey(credentialsId: 'python-app-server', keyFileVariable: 'MY_SSH_KEY')]) {
                    sh "ssh -i $MY_SSH_KEY python-app-server@${APP_NODE_IP} \"cd /home/python-app-server/deployments/Python; chmod 755 start."
                }
            }
        }

        post {
            success {
                echo 'Job completed successfully'
            }
            failure {
                echo 'Job failed'
            }
        }
    }
}
```

CICD for Python Application (incl Ansible deployment)

(13 of 16)

20. Reviewing the python_deployment.yml file

- Unzip the archive to expected location file
- database_ip is passed to the playbook as an extra variable, mapped from the DB_NODE_IP field, and added into the connection.ini file
- pip install from the requirements.txt

```
> CloudOps_Module4_Class9_Repo3_Python_ContinuousDelivery > ! python_deployment.yml
---
- name: Install httpd
  hosts: appservers
  remote_user: python-app-server
  become: false
  ...
  ...
  vars:
    database_ip: "22.22.22.22" # default placeholder value
  ...
  tasks:
    - name: Unarchive the application code
      unarchive:
        src: ~/deployments/gradesApp.zip
        dest: ~/deployments
        remote_src: yes
    ...
    - name: Inject Database IP into the connections file, so that the application can connect
      ansible.builtin.replace:
        path: /home/python-app-server/deployments/Python/connection.ini
        regexp: '\b(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})\b'
        replace: '{{ database_ip }}'
    ...
    - name: Install specified python requirements
      ansible.builtin.pip:
        requirements: /home/python-app-server/deployments/Python/requirements.txt
```

CICD for Python Application (incl Ansible deployment)

(14 of 16)

21. Save the job and click on 'Build Now'.

22. The job will fail first time as you don't have the opportunity to pass the app server and database server IPs until the pipeline code has been retrieved from GitHub.

23. Refresh the page after the first failure, which will change Build Now to Build with Parameters.

24. Enter the required IPs and click Build. It should succeed second time.

Pipeline Python_Deployment

This build requires parameters:

APP_NODE_IP

IP address of Python App Server

33.33.33.33

DB_NODE_IP

IP address of DB Server

44.44.44.44

▶ Build

Cancel

Stage View



CICD for Python Application (incl Ansible deployment)

(15 of 16)

25. Return to the command/Putty window for the app server.

26. Enter a curl command to call the Python API locally.

`curl http://127.0.0.1:5000/listStudentsByCourseID?course_ID=3`

```
[python-app-server@ip-10-0-1-53 .ssh]$ curl http://127.0.0.1:5000/listStudentsByCourseID?course_ID=3
{
  "response": "\n{\n    \"student_ID\": 1,\n    \"first_Name\": \"Ethan\", \n    \"surname\": \"Smith\", \n    \"date_Of_Birth\": \"2000-05-11\"\n}\n{\n    \"student_ID\": 2,\n    \"first_Name\": \"Olivia\", \n    \"surname\": \"Lee\", \n    \"date_Of_Birth\": \"1999-06-12\"\n}\n{\n    \"student_ID\": 6,\n    \"first_Name\": \"Emma\", \n    \"surname\": \"Chen\", \n    \"date_Of_Birth\": \"2001-05-11\"\n}"
}
[python-app-server@ip-10-0-1-53 .ssh]$
```

27. The application has been deployed, connected to the database and is running successfully.

CICD for Python Application (incl Ansible deployment)

(16 of 16)

Agent Python-Node

❗ Disk space is too low. Only 0.474GB left on /tmp.

Labels

python-node

Projects tied to Python-Node

None

28. As the instance size is micro for the Python node, a health check warning is displayed. The action to take is to update the Node Monitoring configuration to a lower Temp Space Threshold and bring the node back online.

Dashboard > Nodes > Node Monitoring

Nodes

Clouds

Node Monitoring

Build Queue

No builds in the queue.

Build Executor Status

Python-Node

offline

Preventive Node Monitoring

☒ Architecture ?

☒ Clock Difference ?

☒ Free Disk Space ?

Free Space Threshold ?
1GB

☒ Free Swap Space

☒ Free Temp Space ?

Free Space Threshold ?
0.3GB

☒ Response Time ?

OK

Apply

Lab 3

Using the CICD for Python Application as a template, create CI and CD pipelines for the application in this example.

The CI pipeline should be the same as the CI pipeline provided, including a final step of creating a zip file for the code. The CD pipeline can ‘deploy’ the code to a folder on Jenkins node on which the pipelines run, or deploy it to a separate node. You can use the Ansible plugin, a shell command calling an ansible playbook, or just use shell commands. The choice is yours.

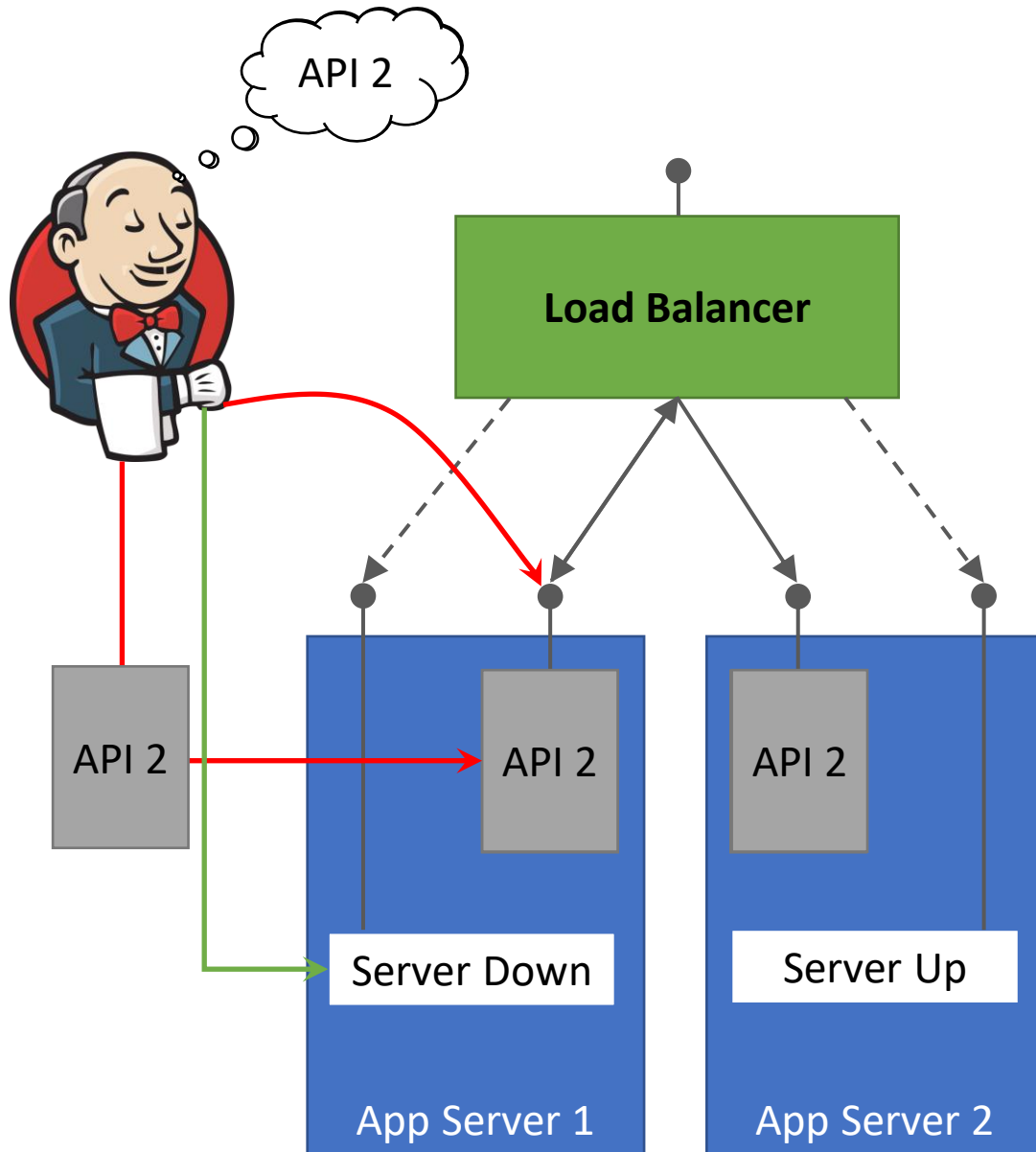


Photo by [Luis Quintero](#) on [Unsplash](#)

Deployment Patterns

Zero Down Time (ZDT) deployment

- Ability to deploy changes to a stateless API where the infrastructure setup is a load balancer in front of two servers
- API availability is 24x7
- Load balancing is using F5 for High Availability, not Capacity.



ZDT - Solution

Load Balancer monitors both API port and server status ports.
Jenkins updates server status during deployment.

1. Start with server 1.
2. Read and remember current deployed version from health check endpoint on server.
3. Set server status to 'Server Drain' for 60 seconds
4. Set server status to 'Server Down'
5. Deploy new API
6. Verify new API
7. Set server status to 'Server Up'
8. Repeat steps 2-7 for server 2



ZDT - Considerations

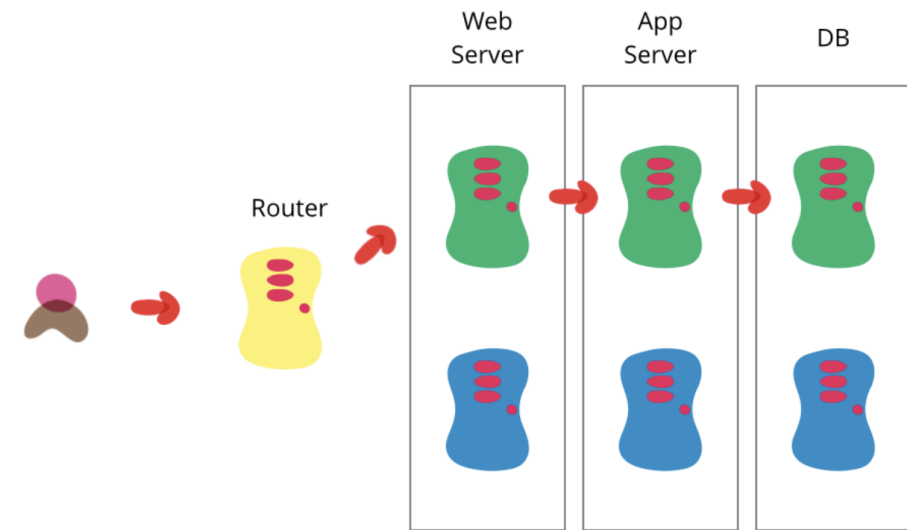
- Good for..
 - Small independent changes like hot fixes or bug defects
- Bad for..
 - Change that impact up or down stream systems
 - Changes that require a lot of post release verification

An automated post verification step is built into the deployment job.

You need confidence in what you are promoting as you have limited opportunity to verify and fix forward with this approach.

Blue Green Deployment

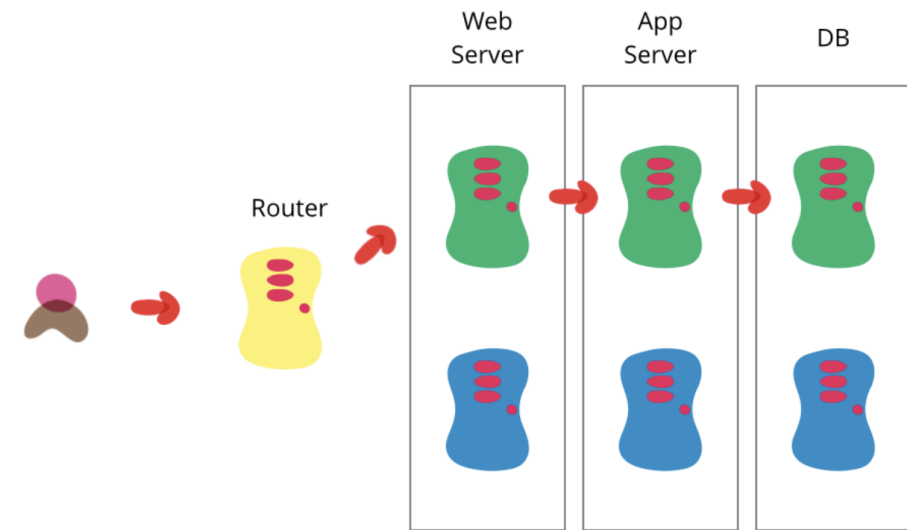
- One of the challenges with automating deployments is that for some applications it needs to be completely seamless.
- With Blue-Green, the new code is running in a staging environment and has been verified. The router configuration is updated to point to the green stack.
- If a rollback is required, the blue stack is still available.
- When the deployment is considered stable, the blue environment becomes staging, until the next production deployment.



<https://martinfowler.com/bliki/BlueGreenDeployment.html>

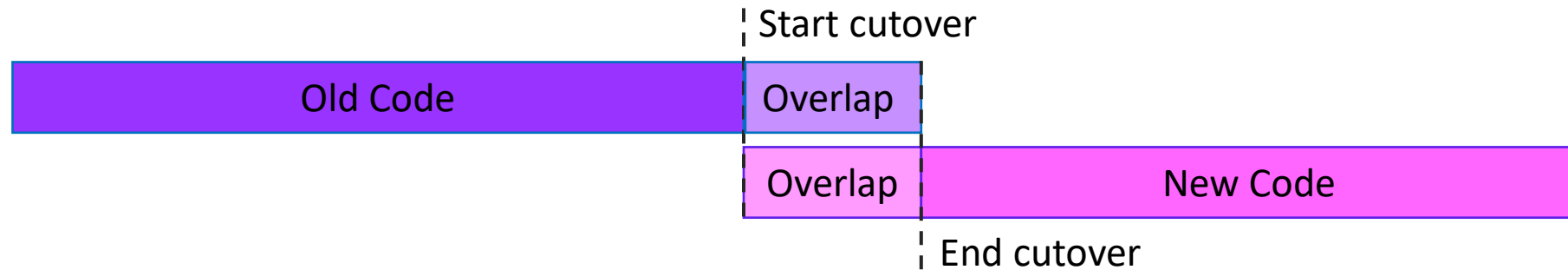
Blue Green Deployment (con't)

- More complex changes can be deployed using this mechanism, as the full application stack can be switched.
- Unlike the ZDT pattern, a lot of validation can be completed on the green stack before it becomes the production instance.
- It can be expensive to maintain too full spec environments.



<https://martinfowler.com/bliki/BlueGreenDeployment.html>

Canary Release



- Allows splitting of live Production traffic between two versions of the application.
- Enables verification of the new version for a small subset of total traffic [typically 10%]
- Deployment can be completed or nullified based on assessment.
- Best suited to a container platform environment, where this mechanism is part of the platform, but can be achieved with other technologies e.g., dynamic updates to Nginx configuration using Hashicorp Consul.

Journey Map

